

Nesta atividade você receberá uma versão inicial de um pequeno jogo chamado Aventura Solo. O programa já possui personagens, inimigos, itens e uma missão simples, porém sua implementação apresenta diversos problemas de organização e modelagem.

Seu objetivo não é construir outro jogo do zero. Você deverá **refatorar e evoluir o código fornecido**, aplicando os conceitos estudados durante a Unidade 1.

Ao final da atividade, o programa deverá continuar funcionando, porém utilizando uma estrutura orientada a objetos mais adequada.

O jogo será executado exclusivamente no terminal. Não é necessário utilizar interface gráfica, banco de dados ou qualquer biblioteca externa.

Contexto do jogo

O jogador controla um aventureiro que deve explorar uma pequena região, enfrentar inimigos, coletar itens e concluir missões.

Inicialmente existirão pelo menos dois tipos de inimigos:

Goblin – inimigo mais fraco, mas que pode realizar ataques rápidos.

Esqueleto – possui maior resistência e causa um tipo diferente de ataque.

O personagem poderá encontrar itens como armas e poções. Algumas missões terão como objetivo derrotar determinados inimigos e entregarão uma recompensa ao jogador.

O jogo termina quando o personagem derrota todos os inimigos definidos para a aventura ou quando seus pontos de vida chegam a zero.

Código inicial

O código abaixo representa a primeira versão do sistema. Ele funciona, mas possui propositalmente vários problemas que deverão ser corrigidos durante a atividade.

Personagem.java

```
import java.util.ArrayList; public class Personagem { public String nome; public int vida;
public int ouro; public ArrayList<Item> inventario = new ArrayList<>(); public
Personagem(String nome) { this.nome = nome; this.vida = 100; this.ouro = 0; } public void
atacar(Inimigo inimigo) { inimigo.vida -= 10; System.out.println( nome + " atacou " +
inimigo.nome + " causando 10 de dano." ); } public void receberDano(int dano) { vida -=
dano; } public void adicionarItem(Item item) { inventario.add(item); } public void
mostrarStatus() { System.out.println("Personagem: " + nome); System.out.println("Vida: " +
vida); System.out.println("Ouro: " + ouro); System.out.println("Itens: " + inventario.size()); } }
```

Item.java

```
public class Item { public String nome; public String tipo; public int valor; public Item(String nome, String tipo, int valor) { this.nome = nome; this.tipo = tipo; this.valor = valor; } }
```

Inimigo.java

```
public class Inimigo { public String nome; public String tipo; public int vida; public int dano; public Inimigo( String nome, String tipo, int vida, int dano ) { this.nome = nome; this.tipo = tipo; this.vida = vida; this.dano = dano; } public void atacar(Personagem personagem) { personagem.receberDano(dano); System.out.println( nome + " atacou " + personagem.nome + " causando " + dano + " de dano." ); } public boolean estaVivo() { return vida > 0; } }
```

Missao.java

```
public class Missao { public String titulo; public String descricao; public boolean concluida; public Inimigo alvo; public Item recompensa; public int ouroRecompensa; public Missao( String titulo, String descricao, Inimigo alvo, Item recompensa, int ouroRecompensa ) { this.titulo = titulo; this.descricao = descricao; this.alvo = alvo; this.recompensa = recompensa; this.ouroRecompensa = ouroRecompensa; } public void concluir(Personagem personagem) { concluida = true; personagem.ouro += ouroRecompensa; personagem.adicionarItem(recompensa); System.out.println( "Missão concluída: " + titulo ); } }
```

JogoAventura.java

```
public class JogoAventura { public static void main(String[] args) { Personagem heroi = new Personagem("Aventureiro"); Item pocao = new Item( "Poção de Cura", "POCAO", 20 ); Inimigo goblin = new Inimigo( "Goblin da Floresta", "GOBLIN", 30, 5 ); Missao missao = new Missao( "Problemas na Floresta", "Derrote o Goblin da Floresta", goblin, pocao, 25 ); while ( goblin.estaVivo() && heroi.vida > 0 ) { heroi.atacar(goblin); if (goblin.estaVivo()) { goblin.atacar(heroi); } } if (heroi.vida > 0) { missao.concluir(heroi); } heroi.mostrarStatus(); } }
```

Sua tarefa

A versão apresentada possui campos públicos, tipos representados por `String`, classes excessivamente genéricas, acesso direto ao estado dos objetos e pouca utilização dos recursos oferecidos pela orientação a objetos.

Você deverá modificar esse projeto progressivamente.

1. **Encapsulamento e proteção do estado – 0,45 ponto.** Todos os atributos das classes deverão deixar de ser públicos. Utilize modificadores de acesso adequados e crie apenas os métodos necessários para consultar ou modificar o estado dos objetos. A vida do personagem e dos inimigos não poderá ser alterada diretamente por outras classes. Crie métodos como `receberDano()`, `curar()` e outros que

considerar necessários. A vida nunca poderá assumir valor negativo. O ouro também não deverá ser alterado diretamente.

2. **Relacionamentos entre objetos e enum – 0,45 ponto.** Organize os relacionamentos entre `Personagem`, `Item`, `Missao` e `Inimigo`. O personagem deverá possuir seu próprio inventário. A missão deverá possuir um inimigo como alvo e uma recompensa. Substitua o `boolean concluida` por um `enum` chamado `EstadoMissao`, contendo pelo menos `PENDENTE` e `CONCLUIDA`. Não utilize `String` para representar estados do sistema quando um `enum` for mais adequado.
3. **Herança e polimorfismo – 0,55 ponto.** Transforme `Inimigo` em uma classe abstrata. Crie pelo menos duas subclasses: `Goblin` e `Esqueleto`. Cada inimigo deverá possuir sua própria implementação de ataque. O programa principal deverá trabalhar com referências do tipo `Inimigo`, sem precisar perguntar se o inimigo é um `Goblin` ou um `Esqueleto`. Evite estruturas como:

```
if (inimigo.tipo.equals("GOBLIN")) { // ... } else if (inimigo.tipo.equals("ESQUELETO")) { // ... }
```

A diferença de comportamento deverá ser resolvida utilizando sobrescrita de métodos e polimorfismo.

4. **Classes abstratas e interfaces – 0,45 ponto.** Transforme `Item` em uma classe abstrata e crie pelo menos `Arma` e `Pocao`. Crie também uma interface chamada `Usavel`, contendo um método semelhante a:

```
void usar(Personagem personagem);
```

A poção deverá implementar essa interface e recuperar pontos de vida do personagem. Pense se todos os itens precisam implementar `Usavel` ou apenas aqueles que realmente podem ser utilizados diretamente.

5. **Exceções e gerenciamento de recursos – 0,40 ponto.** Crie pelo menos uma exceção personalizada relacionada às regras do jogo. Um exemplo seria `ItemNaoEncontradoException`, lançada quando o jogador tentar utilizar um item que não existe no inventário. Essa situação deverá ser tratada com `try` e `catch`. O programa também deverá tratar pelo menos uma exceção não verificada, como uma entrada inválida digitada pelo jogador. Caso utilize `Scanner`, utilize-o com `try-with-resources`.
6. **Collections, generics, equals e hashCode – 0,50 ponto.** Crie uma classe genérica de inventário:

```
public class Inventario<T> { // implementação }
```

Ela deverá utilizar internamente uma coleção adequada e possuir operações para adicionar, remover, procurar e listar elementos. O personagem deverá utilizar algo semelhante a:

```
Inventario<Item>
```

O jogo deverá utilizar pelo menos mais uma estrutura do Collections Framework, como `List`, `Set` ou `Map`, para organizar inimigos, missões ou outras entidades. Implemente `equals()` e `hashCode()` em pelo menos uma entidade na qual a comparação lógica entre objetos seja necessária.

7. **Integração do jogo – 0,20 ponto.** Depois das refatorações, implemente uma pequena aventura jogável no terminal. O jogador deverá conseguir visualizar seu estado, enfrentar inimigos e utilizar pelo menos uma poção. O jogo deverá possuir pelo menos um Goblin e um Esqueleto. Quando todos os objetivos forem concluídos, deverá ser apresentada uma mensagem de vitória. Caso a vida do personagem chegue a zero, deverá ser apresentada uma mensagem de derrota.

Resultado esperado

Não existe uma única implementação correta para o projeto. Diferentes soluções poderão ser utilizadas desde que os conceitos solicitados sejam respeitados.

Uma execução possível poderia apresentar algo semelhante a:

```
=== AVENTURA SOLO === Herói: Kael Vida: 100 Ouro: 0 Você encontrou: Goblin da Floresta 1 - Atacar 2 - Usar item 3 - Ver inventário 4 - Ver status Escolha: 1 Kael atacou Goblin da Floresta. Dano causado: 12 Goblin da Floresta atacou Kael. Dano recebido: 5 Vida de Kael: 95 Vida do Goblin: 18
```

Depois de derrotar um inimigo:

Goblin da Floresta foi derrotado. Missão concluída: Problemas na Floresta Recompensa: 25 moedas Poção de Cura

Ao concluir a aventura:

```
===== AVENTURA CONCLUÍDA
===== Inimigos derrotados: 2 Missões concluídas: 2 Ouro obtido: 60 Parabéns, aventureiro!
```

Não é necessário implementar sistemas adicionais como níveis, experiência, mapas, equipamentos complexos, banco de dados ou interface gráfica.

O objetivo principal desta atividade é demonstrar a capacidade de **organizar, refatorar e evoluir um programa Java utilizando orientação a objetos**.